

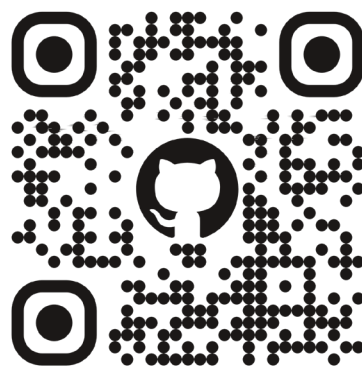
美しさに拘るプログラマー

プロフィール

名前 : なかだ いおり 仲田 伊織
学校 : 福岡情報ITクリエイター専門学校
学科 : ゲーム・クリエイター学科
メール : nkd10121@gmail.com
GitHub : <https://github.com/nkd10121>
Youtube : <https://www.youtube.com/@nkd10121>

スキル

C/C++	: 2年	HLSL	: 半年
C#/Unity	: 2年	Photoshop	: 2年
GitHub	: 2年	Excel	: 1年



GitHubプロフィール
QR



自己PR

今までの**経験**

百人一首▶



ゲーム制作サークルの
「**ふゲだく!**」に所属している



◀ダンス

卓球▶

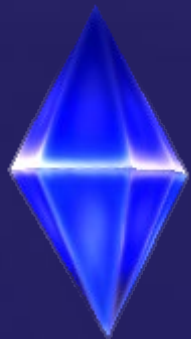


▲イベント出展時の様子

◀サークル活動の様子

**MONSTER
MUST
DIE!!**

最終制作作品



最終制作作品



作品名 : Monster Must Die!
ジャンル : 3Dアクションタワーディフェンス
開発環境 : C++/DxLib、Unity、HLSL、Effekseer
対応機種 : Windows
制作時期 : 2024/10/10 ~
制作期間 : 約5か月
制作人数 : 1人
担当 : モデル、UI、サウンド以外

GitHubURL :
<https://github.com/nkd10121/MonsterMustDie>



動画URL :
<https://youtu.be/er6HSM8xqhl>



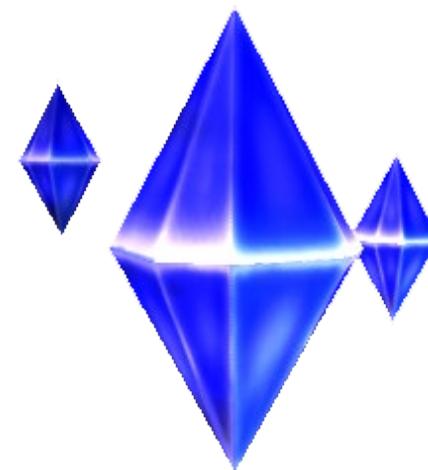
ゲーム概要

MONSTER
MUST
DIE!



床や壁に罠を設置して

迫りくるモンスターから**クリスタル**を守り切れ!



このゲームの面白さ！①

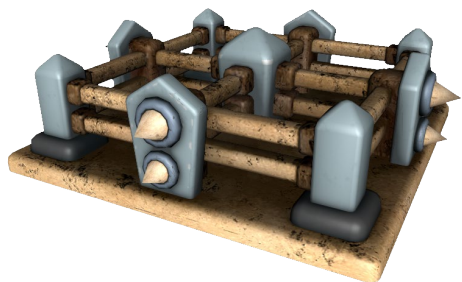
MONSTER
MUST
DIE!

自分だけの攻略法！

罠の配置だけでも

- ・ 罠同士の相性を活かした配置
- ・ マップの構造を活かした配置
- ・ まとめて倒すための配置

など様々な攻略ができる！



このゲームの面白さ！②

MONSTER
MUST
DIE!

タワーディフェンスだけど

ゲーム中でも忙しい！



プレイヤーができること

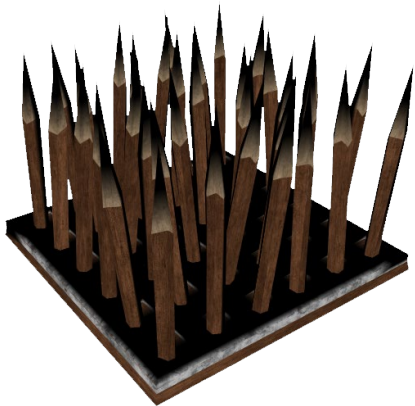
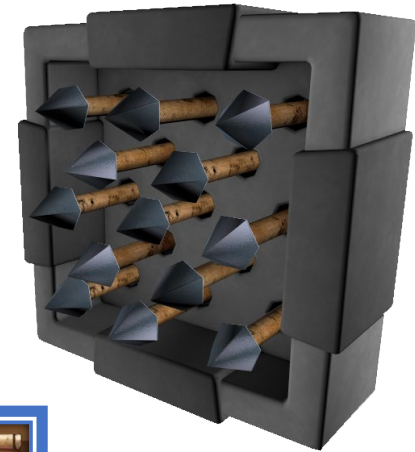
- ・マップを移動
- ・射撃による攻撃
- ・罠の追加設置/売却
- ・敵を罠に誘導

「**見てるだけ**」じゃなく、ずっと**考え、動き、戦う**からこそ忙しくて面白い！

MonsterMustDie !

MONSTER
MUST
DIE!

ワンオペで
モンスターを殲滅せよ！



技術紹介

```
myBig() :  
base()
```

キャラクター名
characterName

```
// <summary>
// 初期化
// </summary>
EnemyBig::
```

モデルが大とすると、コストを少し上げ、

```
ms.y += kPosOffsetY;
```

```
ceManager::GetInstance().GetHandle("M_END
```

```

        設定
        collisionCapsuleRad
        collisionCapsuleSize;

```

[illegible]

```

    = Collidable::AddCollider(MyLib::Collider);
    col = dynamic_cast<MyLib::Collider>(p);
    radius = kCollisionRadius;
    size = kCollisionSize;
}

```

コンストラク
</summary>
Normal Ener
enemyF

```
ColliderBase:  
void* (*collide
```

設定
EnemyNorm

Init()

```
ルを取得  
= ResourceManager::GetInstance
```

```
radius = kCollisionCapsuleRadius;  
size = kCollisionCapsuleSize;
```

```
init();
```

罣設置処理

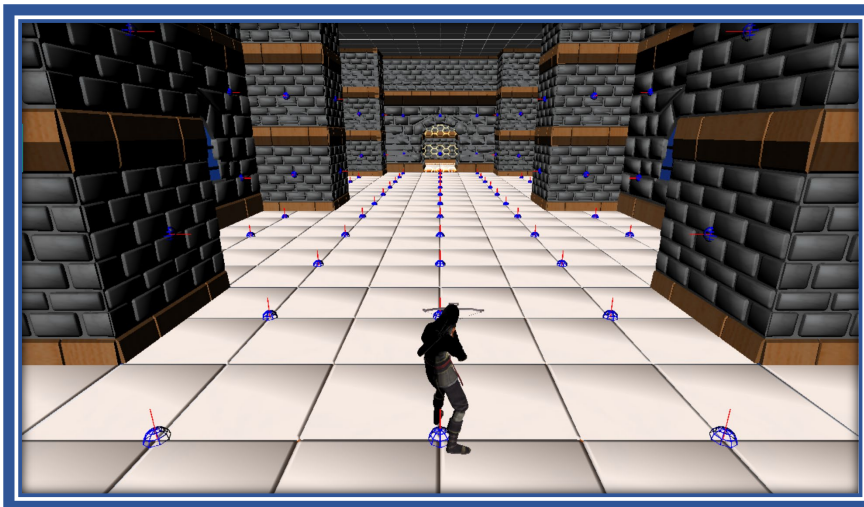
罣を設置するために「罣設置ポイント」をマップに配置している。

【問題点】

元々は画像①のような間隔で置いておき、
1つのポイントにつき1個の罣を置けるようにしていたが、
プレイヤーが「ここに置きたい」と思った場所に置けない。

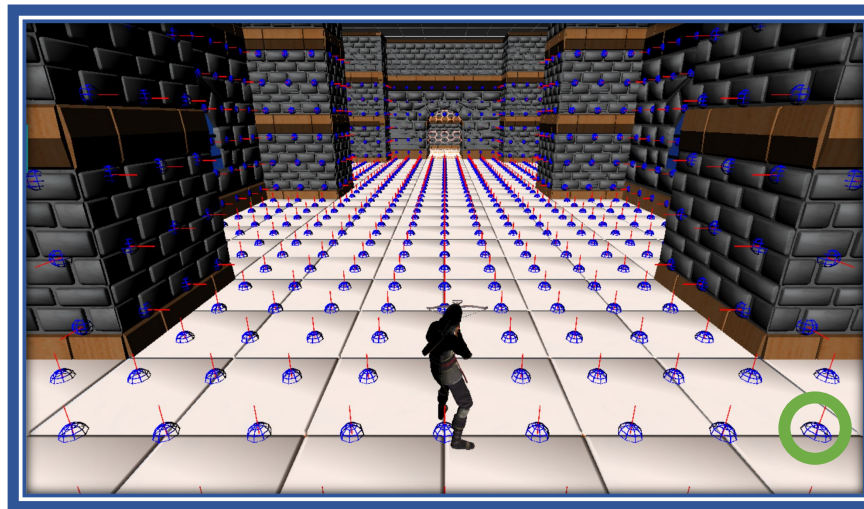
【解決案】

画像②のように配置し、
置きたいと思ったポイントとその周辺のポイントの合計9つのポイントにつき1個の罣を置けるようにすることで、
プレイヤーが思った場所に罣を置けるようにできた。



▲画像①(改善前)

▼画像② (改善後)



(座標) (法線)
青い球と赤い線のセットで1つの罣設置ポイント

シェーダー

守るオブジェクトであるクリスタルに

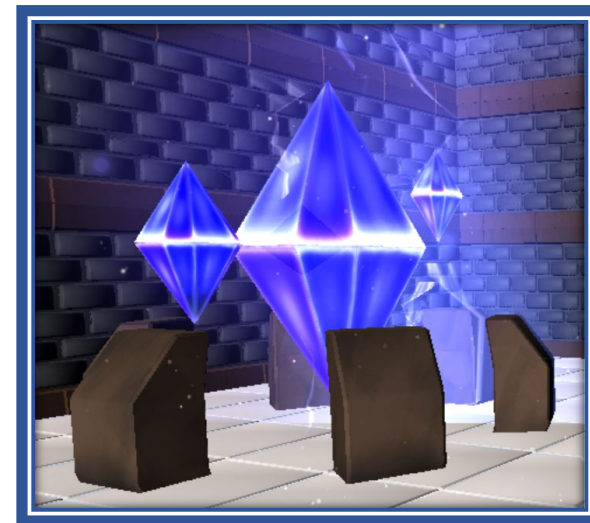
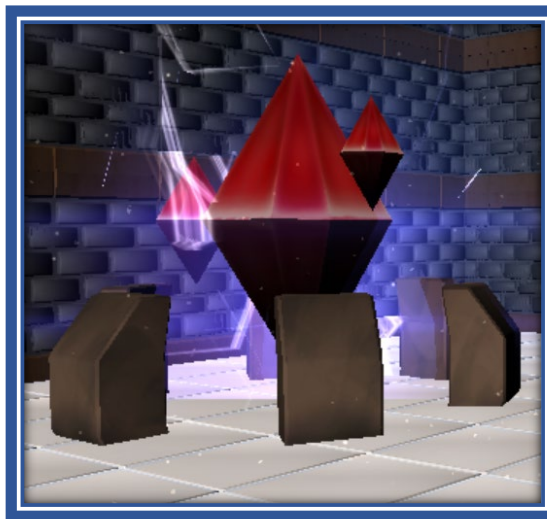
- ・ノイズアニメーションシェーダー
- ・プレイヤー接近時に半透明化するシェーダー
を実装しました。

その結果、

存在感の向上

プレイの快適さの向上

が実現できた。



シェーダー実装前



シェーダー実装後



当たり判定の最適化

ゲームの性質上、複数の当たり判定を持つオブジェクト
が大量に出現する。

処理が重くなる懸念点を

条件を満たしていたら計算をスキップすることで対策した。

▼当たり判定の計算をしない相手のオブジェクトを事前に設定

```

/// <summary>
/// コンストラクタ
/// </summary>
TrapBase::TrapBase():
    ObjectBase(Collidable::Priority::Low, GameObjectTag::Trap),
    m_status(),
    m_isExist(false),
    m_isAttack(false),
    m_trapName(),
    m_isPreview(false),
    m_angle(0.0f)
{
    //敵以外のオブジェクトとは当たり判定をとらない
    AddThroughTag(GameObjectTag::Crystal);
    AddThroughTag(GameObjectTag::Player);
    AddThroughTag(GameObjectTag::SwarmEnemy);
    AddThroughTag(GameObjectTag::Portion);
    AddThroughTag(GameObjectTag::PlayerShot);
    AddThroughTag(GameObjectTag::Trap);
}
    
```

Object/Trap/TrapBase.cpp

▼当たり判定の計算前に相手のタグを確認

```

//どちらかのオブジェクトが相手のオブジェクトタグを無視するようになっていたら当たり判定の計算対象に追加せずに次へ
if (obj1->IsThroughTarget(obj2) || obj2->IsThroughTarget(obj1))
{
    continue;
}
    
```

MyLib/Physics.cpp

▼敵同士の押出判定以外の計算を省略

```

//敵同士かつ、どちらかが通常の当たり判定じゃないときは次に行く
bool isContinue = false;
if (objA->GetTag() == GameObjectTag::Enemy && objB->GetTag() == GameObjectTag::Enemy)
{
    if (colA->collideTag != MyLib::ColliderBase::CollisionTag::Normal ||
        colB->collideTag != MyLib::ColliderBase::CollisionTag::Normal)
    {
        isContinue = true;
    }
}
if (isContinue)
{
    continue;
}
    
```

MyLib/Physics.cpp

▼当たり判定の計算前に距離を計算して計算するか判断

```

//もし規定距離より比較する二つの大きさの合計が大きければ問答無用で追加する
if (maxDistance < (lengthA + lengthB)*(lengthA + lengthB))
{
    // 判定リストに追加
    ret.push_back(obj1);
    ret.push_back(obj2);
    isAdd = true;
    break;
}

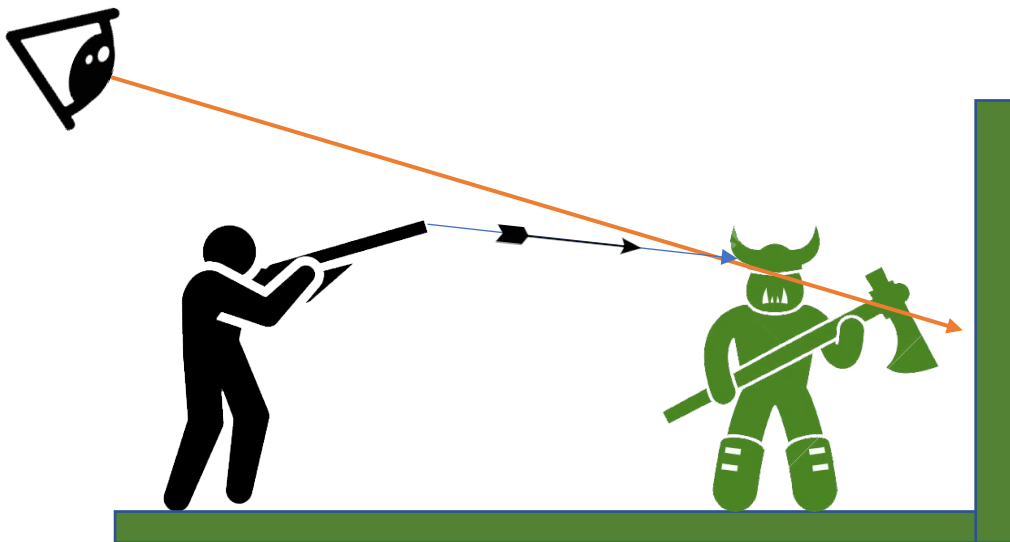
//もし2つのオブジェクトの距離が規定距離より短ければ当たり判定の計算を行う
if ((pos1 - pos2).SqLength() < maxDistance)
{
    // 判定リストに追加
    ret.push_back(obj1);
    ret.push_back(obj2);
    isAdd = true;
    break;
}
    
```

MyLib/Physics.cpp

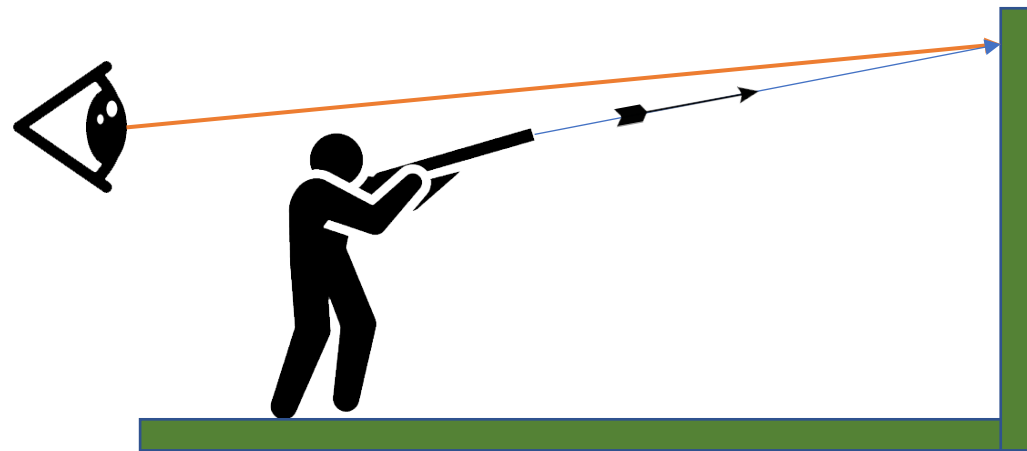
プレイヤーの攻撃

プレイヤーの遠距離攻撃を狙ったところに飛ばすために**レイキャスト**を使用した。

カメラから飛ばしたレイが**敵**や**マップ**と当たっていたら、レイと当たった座標めがけてプレイヤーが射撃する。



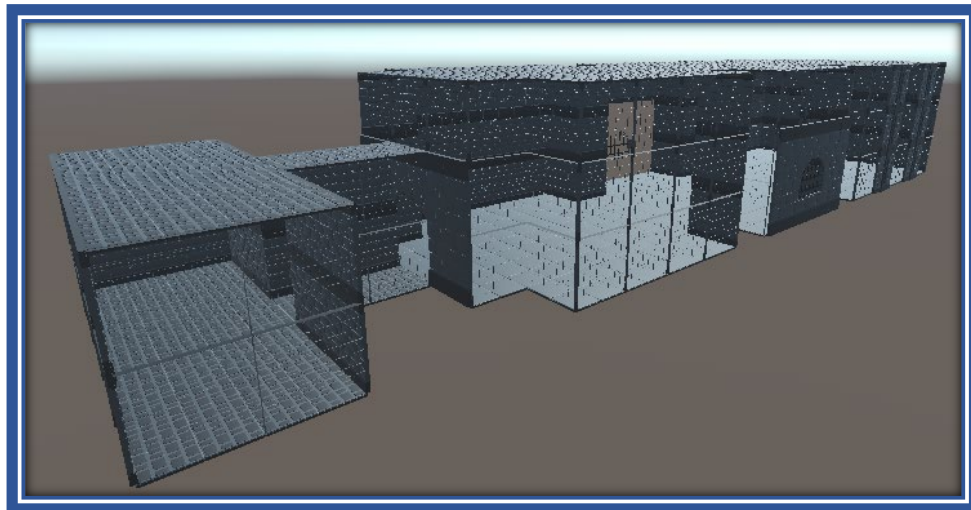
▲**敵**を見ているとき



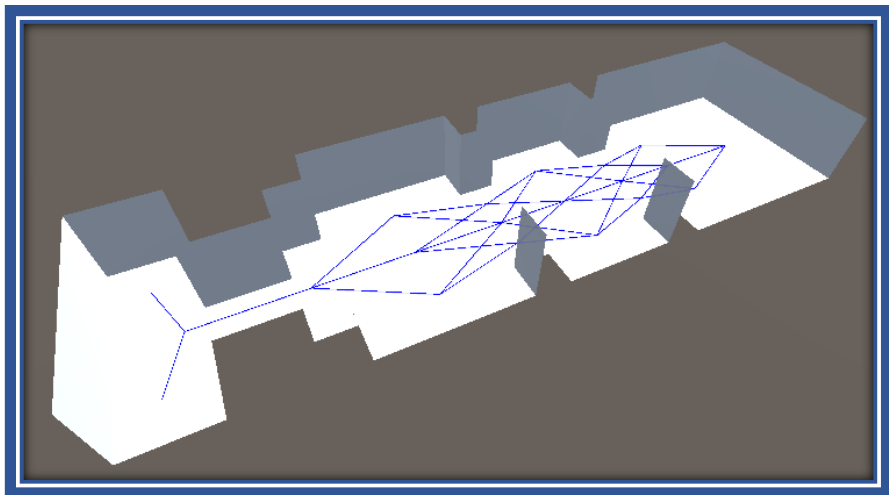
▲**マップ**を見ているとき

マップ制作

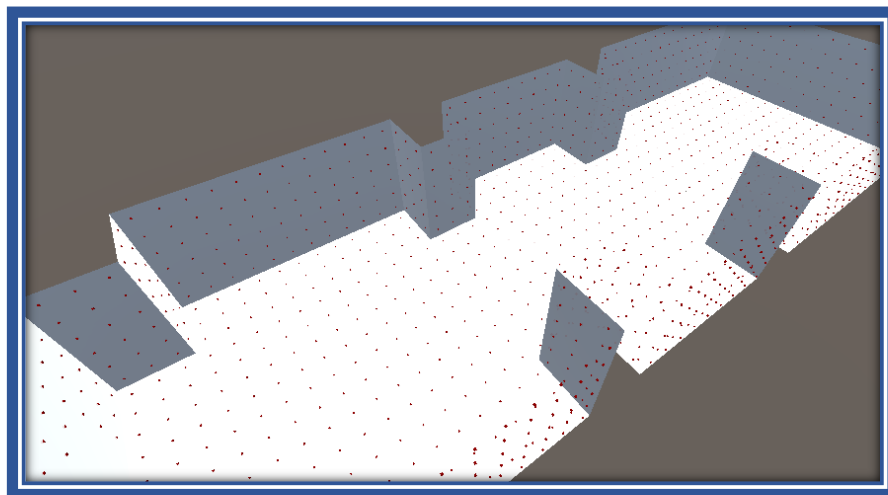
Unityを**マップ制作ツール**として使用。
ステージの制作や変更、レベルデザインの調整がしやすくなった。



▼青い線が敵の移動ルート



▼赤い点が罠設置ポイント



▲マップ構造

▲白いオブジェクトが簡易当たり判定▲

外部ファイル化

- ・キャラクターのステータス
- ・各ステージ情報
- ・敵の出現情報

を**外部ファイル**で**管理**している。

▼敵の出現情報

出現フェーズ	敵ID	出現秒数
1	EnemyNormal	0
1	EnemyNormal	0.5
1	EnemyNormal	1
1	EnemyNormal	1.5
1	EnemyNormal	7.5
1	EnemyNormal	8
1	EnemyNormal	8.5
1	EnemyNormal	9
1	EnemyNormal	10
2	EnemyNormal	0
2	EnemyNormal	0.5
2	EnemyNormal	1
2	EnemyNormal	1.5
2	EnemyNormal	2
2	EnemyBig	2.5

▼ステータス

名前	体力	攻撃力	防御力	移動速度	ドロップポイント	クリスタル与ダメージ
Player	200	12	0	0.5	0	0
EnemyNormal	75	15	0	0.36	20	1
EnemyFast	35	0	0	0.54	15	1
EnemyBig	120	25	2	0.3	30	3

名前	攻撃力	索敵範囲	攻撃範囲	クールタイム	設置コスト	設置場所
Spike	25	6.5	8	260	350	0
ArrowWall	35	5.5	8	360	600	1
Frame	14	6	8	600	800	0
Cutter	6	6	8	0	300	0
IronSnare	0	6	8	0	450	0
IronImpact	50	6	8	800	700	1

▼各ステージ情報

ステージIdx	ステージID	通常当たり判定ID	敵当たり判定ID	フェーズ数	初期震ポイント	初期クリスタルHP	目標クリアタイム	ポーションがドロップする最大数	ステージ名	ミニマップID
0	stage1	M_STAGECOLLISION1	M_E_STAGECOL	3	1800	20	14400		3 ステージ1	I_STAGE1MINIMAP
1	stage2	M_STAGECOLLISION2	M_E_STAGECOL	3	1800	20	14400		3 ステージ2	I_STAGE2MINIMAP
2	stage1	M_STAGECOLLISION1	M_E_STAGECOL	3	1800	20	14400		3 ステージ3	I_STAGE1MINIMAP

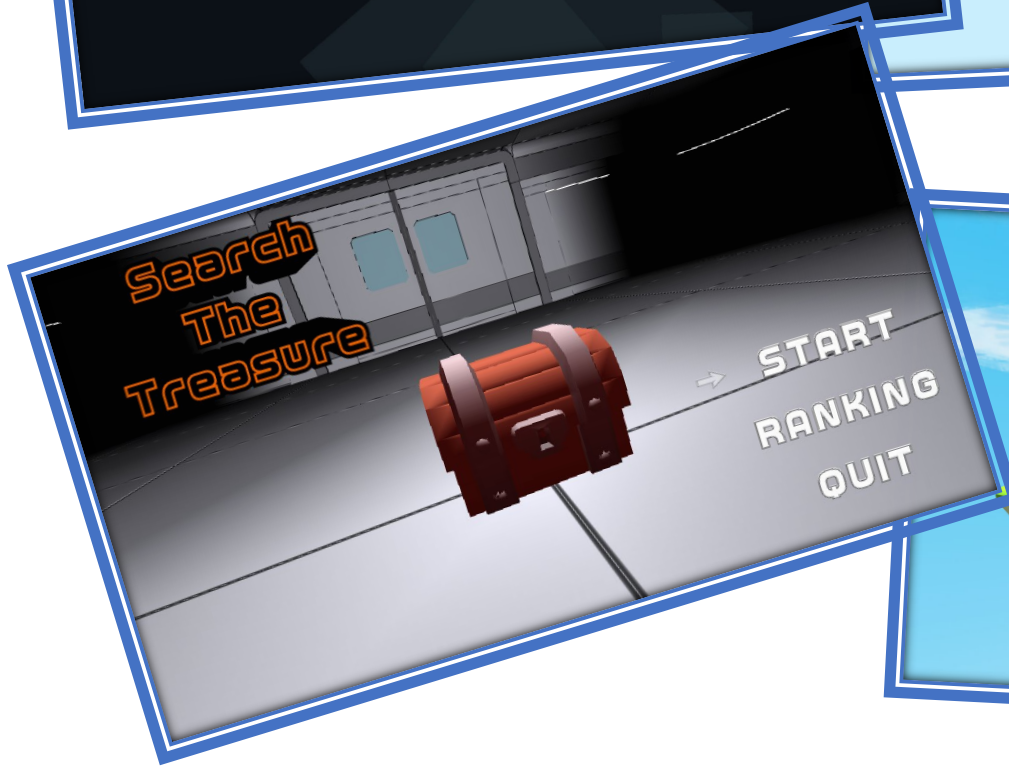
まだまだゲームプレイが単調！

上記の問題を解決するために

- ・ 罾や敵、マップのバリエーションの追加
- ・ 罾周りの仕様の追加
- ・ オンラインランキングの実装

に注力していく予定

その他制作物



その他作品①

チーム制作

Kawaii Survival



作品名 : KawaiiSurvivor
ジャンル : 3Dアクション 開発環境 : C#/Unity
制作時期 : 2023/10 ~ 2024/1
制作人数 : プログラマー 2名
担当箇所 : プレイヤー、カメラ、ボス

頑張ったこと・学んだこと

- ・担当部分の実装
- ・チーム制作の難しさ
- ・3Dゲーム制作の難しさ



作品名 : SpinningTop
ジャンル : 3D押し出し対戦アクション
開発環境 : C#/Unity
制作時期 : 2024/5 ~ 2024/7
制作人数 : プログラマー 3名
担当箇所 : プレイヤー、ローカルマルチ、ステージ

頑張ったこと・学んだこと

- ・プレイヤー挙動
- ・マルチプレイの対応
- ・チーム制作の進捗管理の重要性

その他作品②

個人制作

Skeleton
Must
Break.

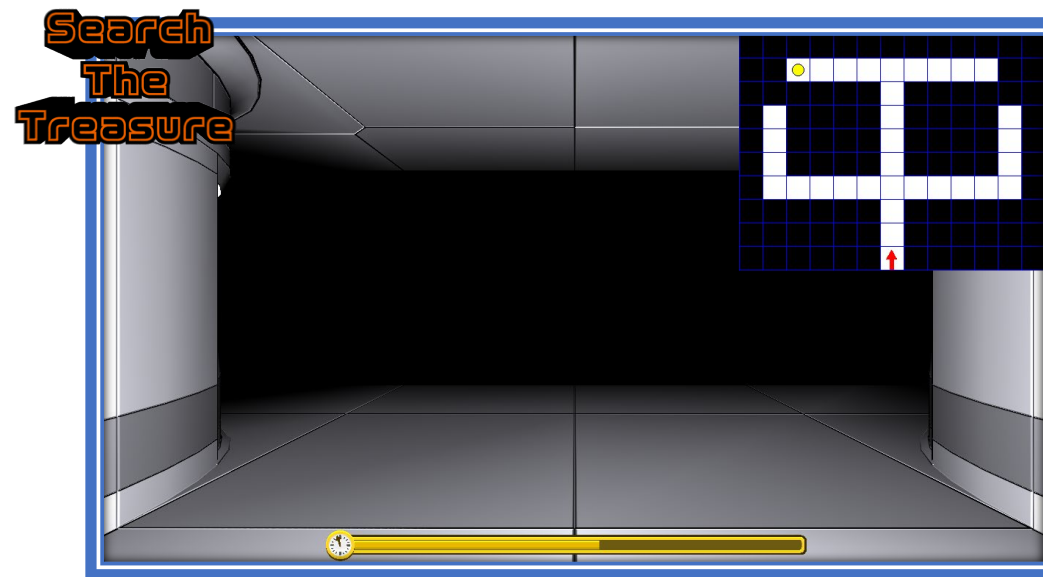
Search
The
Treasure

作品名 : SkeletonMustBreak!
ジャンル : 3Dアクションタワーディフェンス
開発環境 : C++/DxLib、Unity、Effekseer
制作期間 : 2024/7 ~ 2024/9
担当箇所 : モデル、UI、サウンド以外

頑張ったこと・学んだこと

- ・3Dの当たり判定
- ・簡易物理挙動
- ・3Dエフェクトの実装
- ・3Dアニメーションの実装
- ・罠の設置

Gitリポジトリ



作品名 : SearchTheTreasure
ジャンル : 3D迷路
開発環境 : C++/DxLib
制作時期 : 2024/4/ ~ 2024/5
担当箇所 : モデル、UI、サウンド以外

頑張ったこと・学んだこと

- ・ミニマップの作成
- ・外部ファイルでステージを制作/実装
- ・ローカルランキングの実装

Gitリポジトリ



その他作品③

個人制作

Jump
Action

Avoidance
of Shapes

作品名 : JumpAction
ジャンル : 3Dアクション
開発環境 : C++/DxLib
制作時期 : 2024/4/10~2024/4/21
担当箇所 : 3Dモデル、サウンド以外

頑張ったこと・学んだこと

- ・ 3Dモデルの扱い方
- ・ 3Dアニメーションの実装

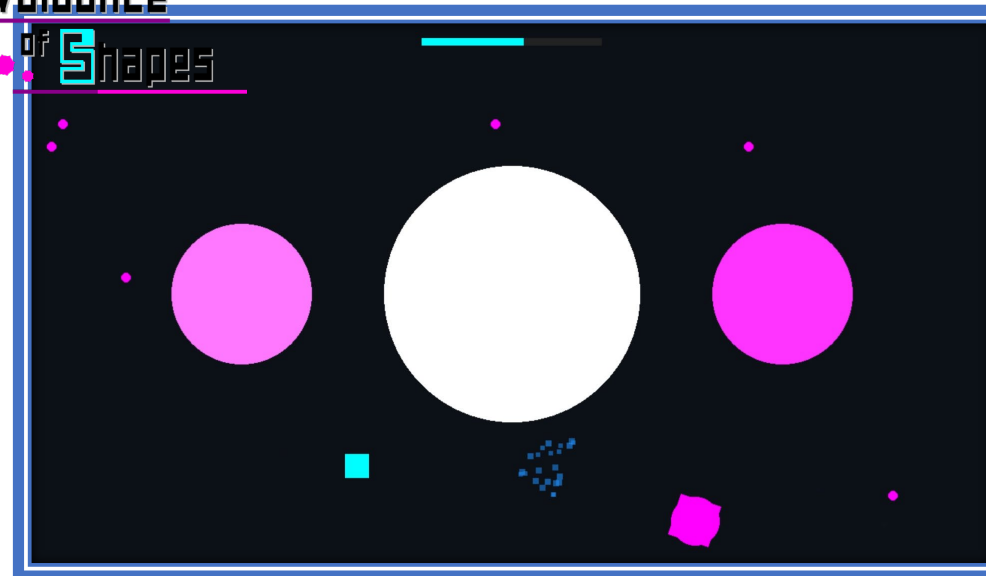
Gitリポジトリ



Jump
Action



Avoidance
of Shapes



作品名 : AvoidanceOfShapes
ジャンル : リズム弾幕アクション
開発環境 : C++/DxLib
制作時期 : 2023/11 ~ 2024/2
担当箇所 : サウンド以外

頑張ったこと・学んだこと

- ・ Dxライブラリでのゲーム制作の流れ
- ・ 2Dの当たり判定
- ・ 2Dエフェクト

Gitリポジトリ



今後の目標

最強の

グラフィックプログラマー

を目指す！

今回の制作でシェーダーに興味を持った。

ゲームの世界観や没入感をより引き立てられるように、基礎からしっかり学び、さまざまなビジュアル表現に挑戦していきたいと考えている。